

Provenance

In DB, Workflows and Preprocessing Pipelines

Khalid Belhajjame
PSL, Université Paris-Dauphine, LAMSADE
kbelhajj@gmail.com

1

Once you have preprocessed your data, created model, used the models, and created a report

Do we just use it and go on?

No, it's not the end –

- How do we refine it over time?
- What if something isn't quite right – can we debug it? justify it in an audit?
- What if we want to repeat the steps to build the model, over new training data?
- What if we are in Europe (GDPR), where someone might **opt out** of consenting for their data to be used – how do we guarantee we aren't using their data?
- Can we reuse the model or the report to evaluate another hypothesis?

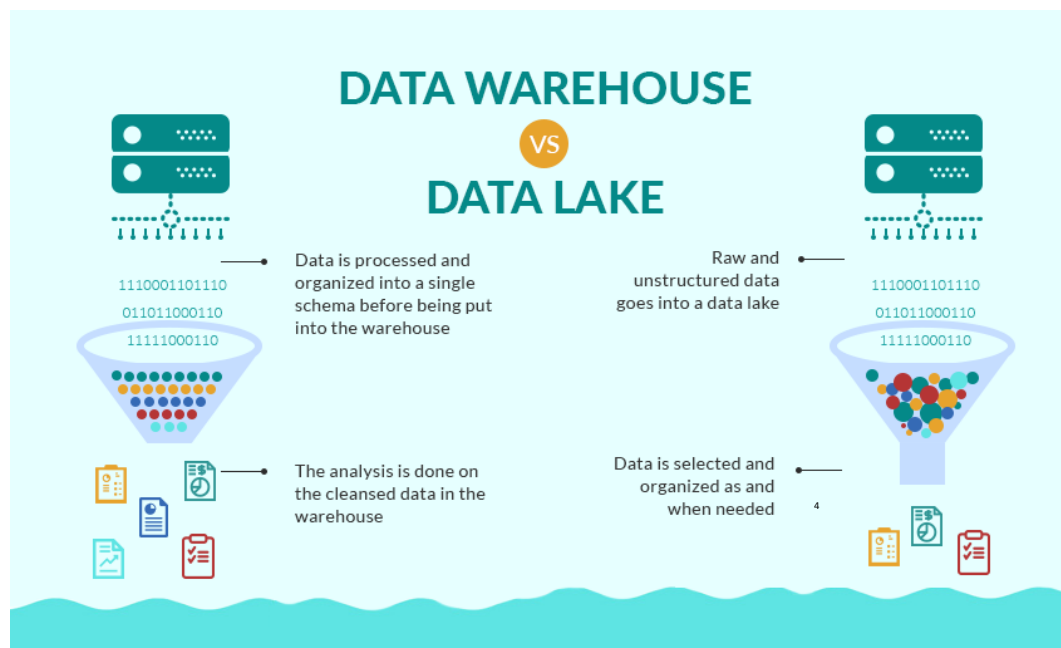
2

Data Lakes to the Rescue

- “Can we keep a very large pool of all of our data” → a “data lake”
- Reports, models, raw data, processed data, data across many versions, ...
- All with a centralized point of access, and with appropriate security controls
- Often: a cloud filesystem like Hadoop HDFS or Amazon S3, with some management capabilities

3

3



<https://www.grazitti.com/blog/data-lake-vs-data-warehouse-which-one-should-you-go-for/>

4

What Can We Do with a Data Lake?

Mainly: it is a central point of persistent storage, for all of the data assets in an organization

- Suitable for direct analytics via Spark or similar big data engines
- Fairly simple to access, and thus make use of, the data
- Also makes it easy to share resources

But we likely still need to find the data, wrangle it, etc.: it's a unified storage system, not a full solution

5

5

Current Data Lakes Leave Lots of Open Problems

- Data lakes are a hot new technology for centralizing data access
- But there are still lots of questions they leave unanswered, for the moment:
 - With thousands or millions of versioned tables and files – how do we *search the data lake* and get appropriate ranked results?
 - How do we *discover relevant new data* published to the data lake?
 - See for example “Alex Bogatu, Alvaro A. A. Fernandes, Norman W. Paton, Nikolaos Konstantinou: Dataset Discovery in Data Lakes. ICDE 2020: 709-720”
 - How do we figure out what's the most important, or most heavily used, data in the lake?
 - This last point motivates *data usage tracking*, i.e., data provenance...

6

6

Data Lakes are not enough!

Do we just use it and go on?

No, it's not the end –

- How do we refine it over time?
- **What if something isn't quite right – can we debug it? justify it in an audit?**
- What if we want to repeat the steps to build the model, over new training data?
- **What if we are in Europe (GDPR), where someone might opt out of consenting for their data to be used – how do we guarantee we aren't using their data?**
- **Can we reuse the model or the report to evaluate another hypothesis?**
- **Is the analysis performed reproducible ?**

7

Capturing File-Level Provenance

8

Conceptually...

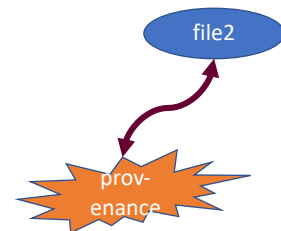
For every derived result, we want to record a “trace” of

- The steps involved in creating the result
- The inputs
- The parameters

```
1. Read file1
2. Sort rows
3. Compute
   median
4. Write to
   file2
```

And from a tools perspective, we want:

- to automatically capture provenance, without programmer intervention
- to collect everything into a “provenance management system”
- to be able to reason about data + provenance

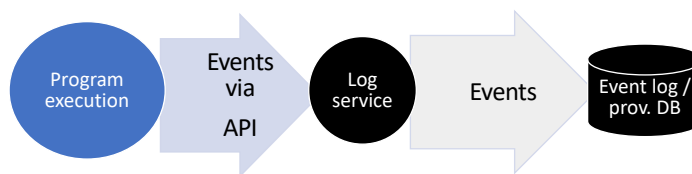


9

A First Cut at Capturing Data Provenance

Provide a service for capturing the “trace”: **log / event messages capturing relationships**

- Opened a file, appended, etc.



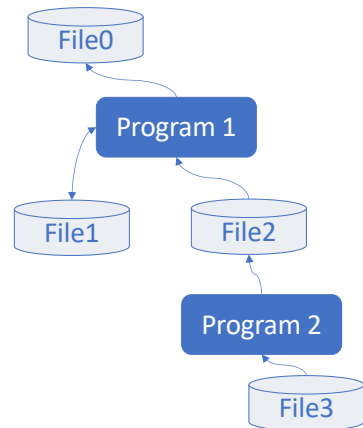
This approach, and variants thereof, have been used in many settings

e.g., PASS for filesystems [Seltzer+ FAST05], Mesquite for phylogenetics (mesquiteproject.org),

Also within Google, Facebook, etc.!

10

An Example



1. Exec **Program 1** with parameters ["File1"]
2. **Program 1** reads File1
3. **Program 1** reads File2
4. **Program 1** writes to File1
5. **Program 1** reads File1
6. **Program 1** writes to File1
7. **Program 1** writes to File0
- ...

11

Issues

- Low-level events are typically *overwhelming* in scale & complexity
- A log *sequentializes* everything, even when order doesn't matter – hard to tell if File1's content is based on File2's content!

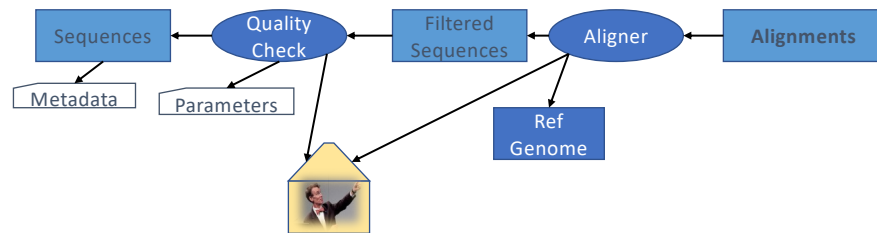
1. Exec **Program 1** with parameters ["File1"]
2. **Program 1** reads File1
3. **Program 1** reads File2
4. **Program 1** writes to File1
5. **Program 1** reads File1
6. **Program 1** writes to File1
7. **Program 1** writes to File0
- ...

12

A More Understandable Model: *Dataflow* Dependencies

“Lift” abstraction to show flow of data, as opposed to sequences

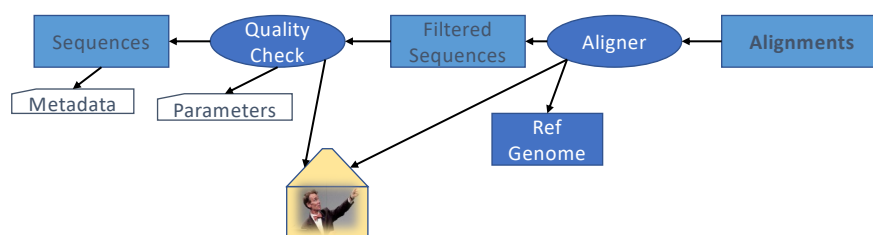
Input files -> computational steps (eg programs, cells) -> output files



We can *annotate* a file with a graph describing the computation steps and inputs

13

Provenance as Relationships



- We can actually take the above structure (a graph), and **map it into a provenance database!!!**
- Now we can use SQL (or other languages) to “trace” the graph for a given file
- This is what many real-world systems do!

14

File-Level Provenance Is Helpful, But Maybe Not Enough

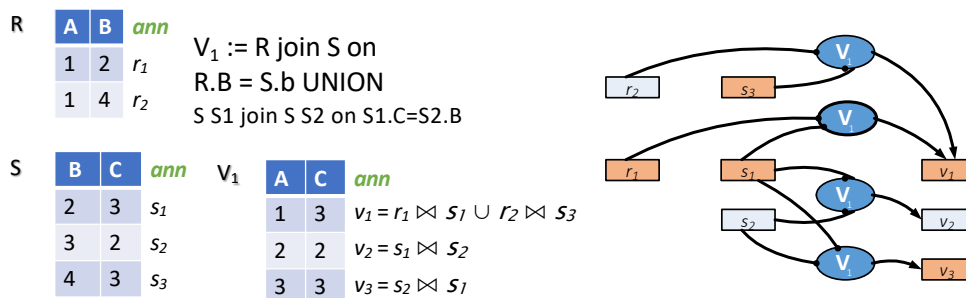
- High-level abstraction makes it much easier to understand derivations
- Standard formats, such as PROV [1], give a format for capturing *entities, agents, activities*
- But: **explains overall outputs, not specific results**
- What if we need to test whether we've included a given (say, GDPR-protected) record? Or to debug one result vs another?

[1] <http://www.w3.org/TR/prov-dm>

15

Provenance in Databases

- Unlike file-level provenance, a number of database systems have been instrumented to capture fine-grained provenance, at the level of the record.
- How does it work?
 - We assign provenance tokens (equiv. node IDs) for each tuple
 - Derived tuples are annotated with expressions reflecting the operations
 - Captures relational operations, PLUS some user-defined operations

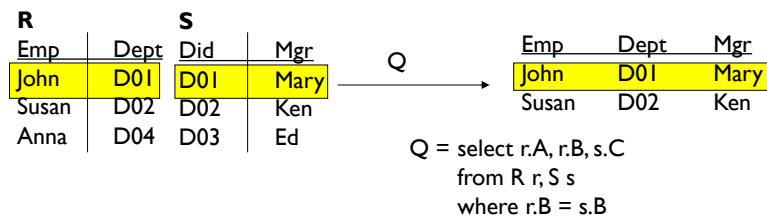


16

Example of Data Provenance

► A typical question:

- For a given database query Q , a database D and a tuple t in the output of $Q(D)$, which parts of D “contribute” to t ?

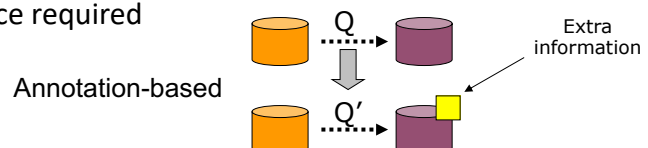


- The question can be applied to attribute values, tables, etc.

17

Two Approaches

- Eager or annotation-based
 - Changes the transformation from Q to Q' to carry extra information
 - Source data not needed after transformation
- Lazy or non-annotation based
 - Q is unchanged
 - Good when extra storage is an issue
 - Recomputation and access to source required



18

Types of Provenance

- Lineage
 - “What is the relevant subset of the input dataset?”
- Why
 - “What DB tuples contribute to the presence of each result tuple?”
- How
 - “By what process is each output tuple produced from the DB instance?”

19

Lineage

- **Lineage** for an output tuple t is a subset of the input tuples which are **relevant** to the output tuple

Agencies			
	name	based_in	phone
t_1 :	BayTours	San Francisco	415-1200
t_2 :	HarborCruz	Santa Cruz	831-3000

ExternalTours				
	name	destination	type	price
t_3 :	BayTours	San Francisco	cable car	\$50
t_4 :	BayTours	Santa Cruz	bus	\$100
t_5 :	BayTours	Santa Cruz	boat	\$250
t_6 :	BayTours	Monterey	boat	\$400
t_7 :	HarborCruz	Monterey	boat	\$200
t_8 :	HarborCruz	Carmel	train	\$90

Q_1 :
 SELECT $a.name, a.phone$
 FROM Agencies a , ExternalTours e
 WHERE $a.name = e.name$
 AND $e.type = 'boat'$

Result of Q_1 :

	name	phone
t	BayTours	415-1200
	HarborCruz	831-3000

Lineage: $\{t_1, t_5, t_6\}$

Problem: Not very precise.

e.g. lineage above does not specify that t_5 and t_6 do not both need to exist.

20

Why provenance

Agencies			
	name	based in	phone
t_1 :	BayTours	San Francisco	415-1200
t_2 :	HarborCruz	Santa Cruz	831-3000

ExternalTours				
	name	destination	type	price
t_3 :	BayTours	San Francisco	cable car	\$50
t_4 :	BayTours	Santa Cruz	bus	\$100
t_5 :	BayTours	Santa Cruz	boat	\$250
t_6 :	BayTours	Monterey	boat	\$400
t_7 :	HarborCruz	Monterey	boat	\$200
t_8 :	HarborCruz	Carmel	train	\$90

Q_1 :
 SELECT $a.name, a.phone$
 FROM Agencies a , ExternalTours e
 WHERE $a.name = e.name$
 AND $e.type='boat'$

Result of Q_1 :

	name	phone
t	BayTours	415-1200
	HarborCruz	831-3000

Witness of t : Any subset of the database sufficient to reconstruct tuple t in the query result.

$\{t_1, t_5\} \quad \{t_1, t_6\}$

Witness basis: Leaves of the “proof tree” showing how result tuple t is generated

$\{\{t_1, t_5\}, \{t_1, t_6\}\}$

21

Why: Query Rewriting

	A	B
t_1	1	2
t_2	1	3
t_3	4	2

Output of
 $Q(I), Q'(I)$:

	A	B
t	1	2
	1	3
	4	2

$Q : Ans(x, y) :- R(x, y).$

$Q' : Ans(x, y) :- R(x, y), R(x, z)$

Why(Q, I, t): $\{\{t_1\}\}$

Why(Q', I, t): $\{\{t_1\}, \{t_1, t_2\}\}$

Minimal witness basis:

Minimal witnesses in the witness basis

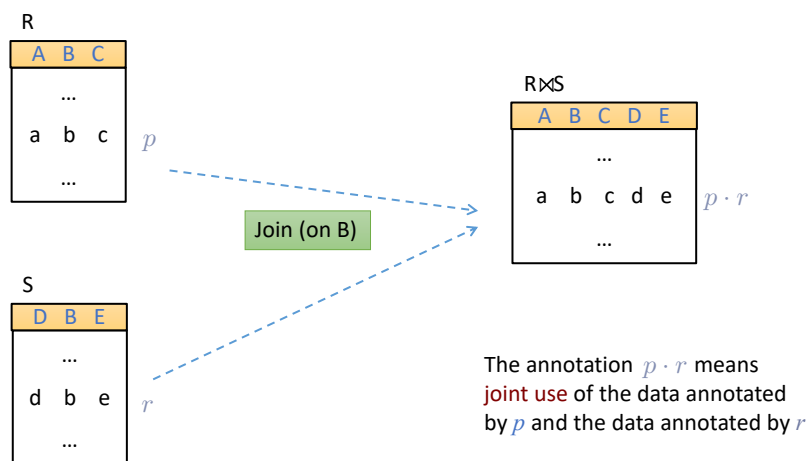
22

How provenance

- Identifies “witness tuples” **and** the operations performed on them to produce each result tuple
- Expresses operations using provenance semirings
 - MERGE (+): union or projection
 - JOIN ($\cdot$): joins

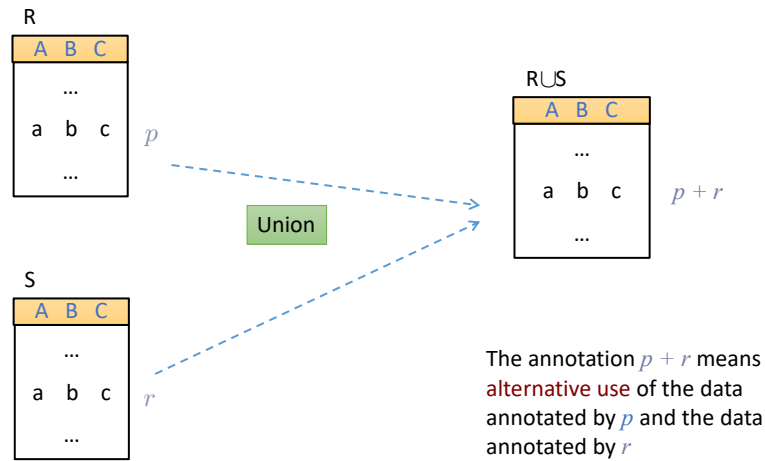
23

Propagating Annotations



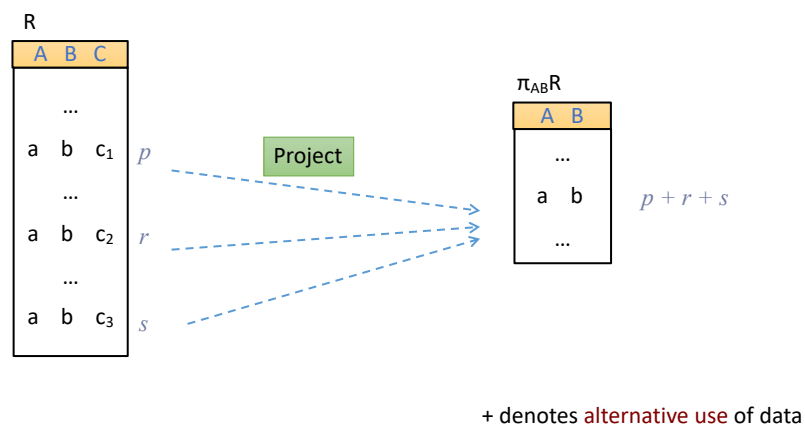
24

Propagating Annotations (2)



25

Propagating Annotations (3)



26

An example (SPJU)

Compute the annotation (how-provenance) of the result of the following query

R

A	B	C	
a	b	c	<i>p</i>
d	b	e	<i>r</i>
f	g	e	<i>s</i>

$$Q = \pi_{AC}(\pi_{AB}R \bowtie \pi_{BC}R \cup \pi_{AC}R \bowtie \pi_{BC}R)$$

27

An example (SPJU)

R

A	B	C	
a	b	c	<i>p</i>
d	b	e	<i>r</i>
f	g	e	<i>s</i>

$$Q = \pi_{AC}(\pi_{AB}R \bowtie \pi_{BC}R \cup \pi_{AC}R \bowtie \pi_{BC}R)$$

A	C	
a	c	$p \cdot p + p \cdot p$
a	e	$p \cdot r$
d	c	$r \cdot p$
d	e	$r \cdot r + r \cdot s + r \cdot r$
f	e	$s \cdot s + s \cdot r + s \cdot s$

28

An example (SPJU)

R

A	B	C
a	b	c
d	b	e
f	g	e

*p**r**s*

$$Q = \sigma_{C=e} \pi_{AC} (\pi_{AB} R \bowtie \pi_{BC} R \cup \pi_{AC} R \bowtie \pi_{BC} R)$$

A	C
a	c
a	e
d	c
d	e
f	e

$$(p \cdot p + p \cdot p) \cdot 0$$

$$p \cdot r \cdot 1$$

$$r \cdot p \cdot 0$$

$$(r \cdot r + r \cdot s + r \cdot r) \cdot 1$$

$$(s \cdot s + s \cdot r + s \cdot s) \cdot 1$$

For selection, multiply with annotation 0 and 1.

29

Applying the laws: **polynomials**

R

A	B	C
a	b	c
d	b	e
f	g	e

*p**r**s*

Q

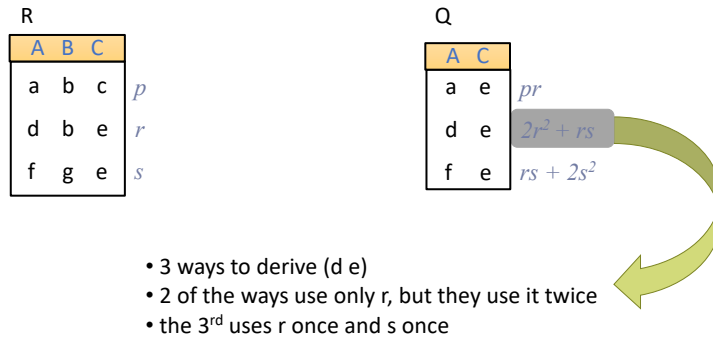
A	C
a	e
d	e
f	e

pr $2r^2 + rs$ $rs + 2s^2$

Polynomials with coefficients in $\mathbb{N}$ and annotation tokens as indeterminates *p*, *r*, *s* capture a very general form of provenance

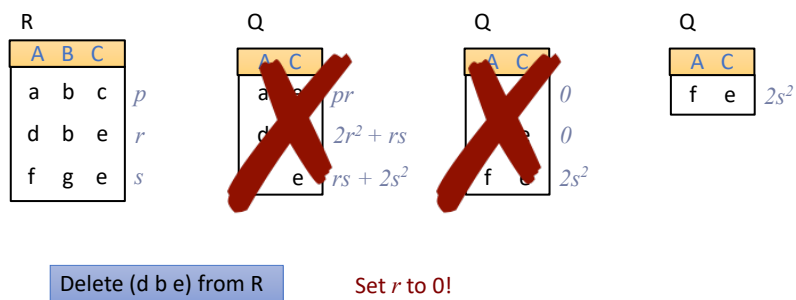
30

How to read this provenance



31

Deletion Propagation



32

Where Provenance

- Where-provenance describes where a piece of data is copied from.
- While why-provenance is about the relationship between source and output tuples, **where-provenance describes the relationship between source and output locations.**
- In the relational setting, a location is simply a column of a tuple in a relation, which precisely refers to a “cell” in a relation.
- The where-provenance of a value that resides in some location l in $Q(D)$ consists of locations of D from which the value in l was copied according to Q .

33

Where Provenance: Example

Agencies			
	name	based_in	phone
t_1 :	BayTours	San Francisco	415-1200
t_2 :	HarborCruz	Santa Cruz	831-3000

ExternalTours				
	name	destination	type	price
t_3 :	BayTours	San Francisco	cable car	\$50
t_4 :	BayTours	Santa Cruz	bus	\$100
t_5 :	BayTours	Santa Cruz	boat	\$250
t_6 :	BayTours	Monterey	boat	\$400
t_7 :	HarborCruz	Monterey	boat	\$200
t_8 :	HarborCruz	Carmel	train	\$90

Q_1 :
 SELECT $a.name, a.phone$
 FROM Agencies a , ExternalTours e
 WHERE $a.name = e.name$ AND
 $e.type = 'boat'$

Result of Q_1 :	
name	phone
BayTours	415-1200
HarborCruz	831-3000

For example, the where-provenance of the value “**HarborCruz**” in the second output tuple in the result of Q_1 is the location (**t_2 , name**) in our example database, since “HarborCruz” was copied from the name attribute of the tuple t_2 in the Agencies relation, according to Q_1 .

34

Where Provenance

- Where-provenance is not invariant under equivalent queries.

Q_1 :
 SELECT a.name, a.phone
 FROM Agencies a, ExternalTours e
 WHERE a.name = e.name
 AND e.type='boat'

Q'_1 :
 SELECT e.name, a.phone
 FROM Agencies a, ExternalTours e
 WHERE a.name = e.name
 AND e.type='boat'

Agencies		
	name	based_in
t_1 :	BayTours	San Francisco
t_2 :	HarborCruz	Santa Cruz

ExternalTours			
	name	destination	type
t_3 :	BayTours	San Francisco	cable car
t_4 :	BayTours	Santa Cruz	bus
t_5 :	BayTours	Santa Cruz	boat
t_6 :	BayTours	Monterey	boat
t_7 :	HarborCruz	Monterey	boat
t_8 :	HarborCruz	Carmel	train

Results of Q_1 and Q'_1

name	phone
BayTours	415-1200
HarborCruz	831-3000

- The only difference between Q_1 and Q'_1 is in the select clause.
- The queries Q_1 and Q'_1 are equivalent in that they produce the same resulting tuples for any given input database.
- However, the where-provenance of the output value "HarborCruz" is different under the two queries.
 - The where-provenance of "HarborCruz" in the output according to Q_1 is the location (t_2 , name)
 - The where-provenance of "HarborCruz" is (t_7 , name)

35

Where Provenance

Q_1 :
 SELECT a.name, a.phone
 FROM Agencies a, ExternalTours e
 WHERE a.name = e.name
 AND e.type='boat'

Q'_1 :
 SELECT e.name, a.phone
 FROM Agencies a, ExternalTours e
 WHERE a.name = e.name
 AND e.type='boat'

Agencies		
	name	based_in
t_1 :	BayTours	San Francisco
t_2 :	HarborCruz	Santa Cruz

ExternalTours			
	name	destination	type
t_3 :	BayTours	San Francisco	cable car
t_4 :	BayTours	Santa Cruz	bus
t_5 :	BayTours	Santa Cruz	boat
t_6 :	BayTours	Monterey	boat
t_7 :	HarborCruz	Monterey	boat
t_8 :	HarborCruz	Carmel	train

Results of Q_1 and Q'_1

name	phone
BayTours	415-1200
HarborCruz	831-3000

- If a value v of a location of an output tuple t is not constructed by a query Q , then it must have been copied from values that reside in some source locations of a witness of t according to Q .
- As a consequence, the where-provenance of v consists of locations that can be found in tuples of the why-provenance of t .
- The why-provenance of t according to Q'_1 is $\{\{t_1, t_5\}, \{t_1, t_6\}\}$.
- The where-provenance of the value "BayTours" at location (t , name) consists of two locations (t_5 , name) and (t_6 , name).

36

Where Provenance: Annotation Propagation

- Given database, annotations on attribute value
- How to annotate query results? (Not the level of a tuple, but instead attribute values)
- Alternative Goal: study **Where** values (data) are copied from
- Rationale: add annotations based on annotations in Where-provenance
 - Copying values $\Rightarrow$ copying annotations

37

Example of an Open Source Database System with provenance Support

- Perm: Perm is an extended PostgreSQL server with support for capturing provenance of SQL queries on fly. Provenance is exposed through several SQL extensions.
 Features: <http://www.cs.iit.edu/~dbgroup/projects/perm.html>
 Github: <https://github.com/IITDBGGroup/perm>
- ProvSQL: Extends Postgres with provenance recording capabilities.
 It support why-provenance, how-provenance, as well as other flavors of provenance
 A demonstration version is available as a docker container
 Github: <https://github.com/PierreSenellart/provsql>

38